Lecture 05.12: Inheritance in OOPs

📌 Executive Summary

Base and derived classes, IS-A relationships, code reuse, the super() proxy object, and overriding base methods.

🔬 Theoretical Foundations & Principles

1. Generalization & Specialization (IS-A)

Inheritance models hierarchical taxonomic relationships where a derived subclass specializes a generalized base superclass:

$$\text{SubClass} \subseteq \text{SuperClass}$$

2. The `super()` Proxy

`super()` returns a proxy object delegating method calls to a parent or sibling class in the Method Resolution Order (MRO). In single inheritance:

$$\text{super}().\text{method}(*\text{args}) \implies \text{BaseClass}.\text{method}(\text{self}, *\text{args})$$

💻 Production Implementation & Code

```
class Vehicle:
    def __init__(self, brand: str, model: str):
        self.brand = brand
        self.model = model

    def describe(self) -> str:
        return f"{self.brand} {self.model}"

class Car(Vehicle):
    def __init__(self, brand: str, model: str, seats: int):
```

```
        super().__init__(brand, model)
        self.seats = seats

    def describe(self) -> str:
        return f"{super().describe()} (Seats: {self.seats})"

c = Car("Toyota", "Camry", 5)
print(c.describe())
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.